

Лекция. Простая анимация с помощью Jetpack Compose

В уроке вы узнаете, как добавить простую анимацию в приложение для Android. Анимация может сделать ваше приложение более интерактивным, интересным и легким для восприятия пользователями. Анимация отдельных обновлений на экране, заполненном информацией, поможет пользователю увидеть, что изменилось.

Существует множество типов анимации, которые можно использовать в пользовательском интерфейсе приложения. Элементы могут появляться и исчезать, перемещаться по экрану или за его пределы, а также интересно трансформироваться. Это помогает сделать пользовательский интерфейс приложения выразительным и удобным в использовании.

Анимация также может придать вашему приложению полированный вид, что делает его элегантным и одновременно помогает пользователю.

Необходимые условия

- Знание Kotlin, включая функции, лямбды и stateless composables.
- Базовые знания о том, как создавать макеты в Jetpack Compose.
- Базовые знания о том, как создавать списки в Jetpack Compose.
- Базовые знания о Material Design.

Что вы узнаете

- Как создать простую анимацию с помощью Jetpack Compose.

Что вы будете создавать

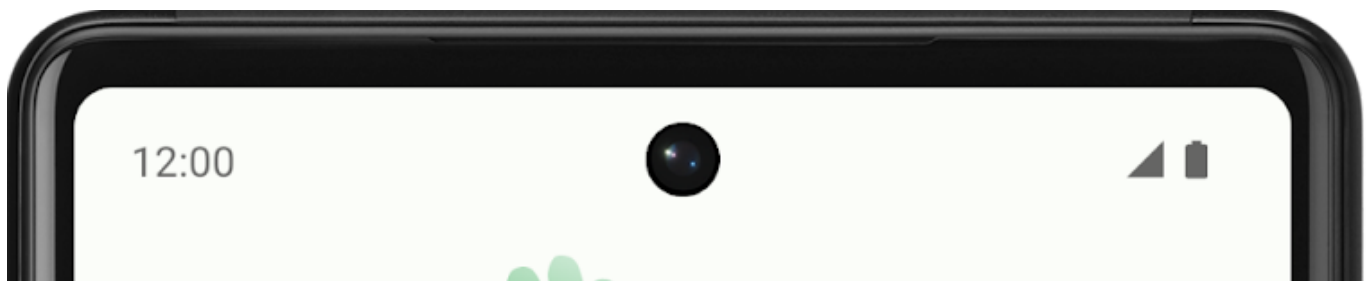
- Вы создадите приложение **Woof** из практической работы по Material Theming with Jetpack Compose и добавите простую анимацию, подтверждающую действие пользователя.

Что вам понадобится

- Последняя стабильная версия Android Studio.
- Подключение к локальному серверу **gogc** для загрузки стартового кода.

Обзор приложения

В практической работе Material Theming with Jetpack Compose вы создали приложение **Woof** с использованием Material Design, которое отображает список собак и информацию о них.





Koda

2 years old



Lola

16 years old



Frankie

2 years old



Nox

8 years old



Faye

8 years old



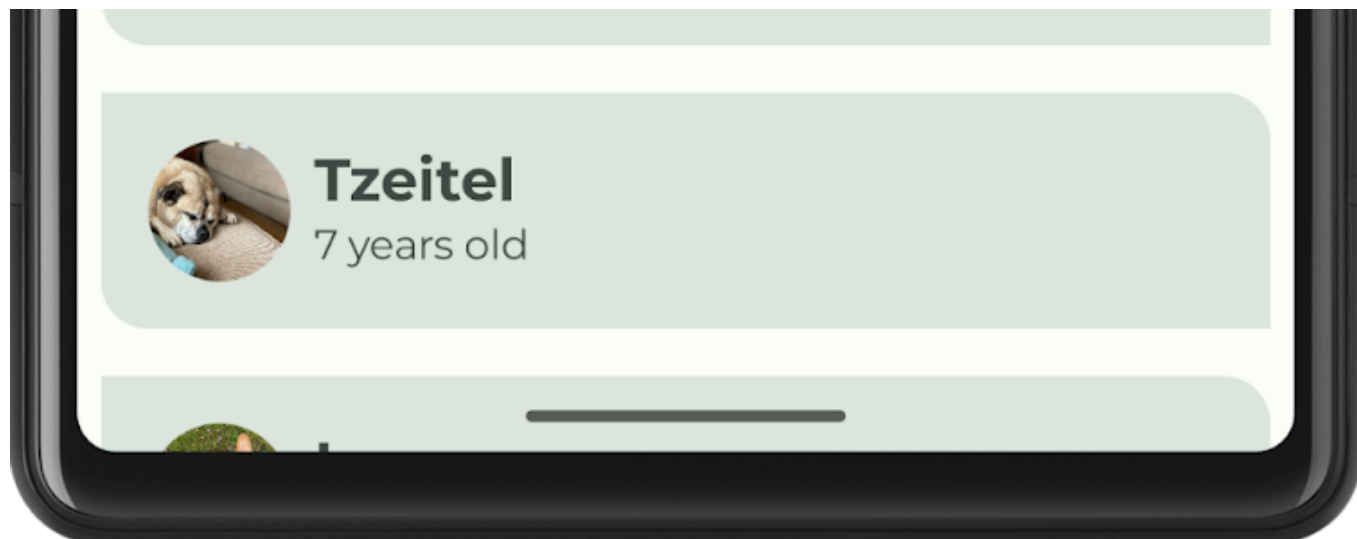
Bella

14 years old

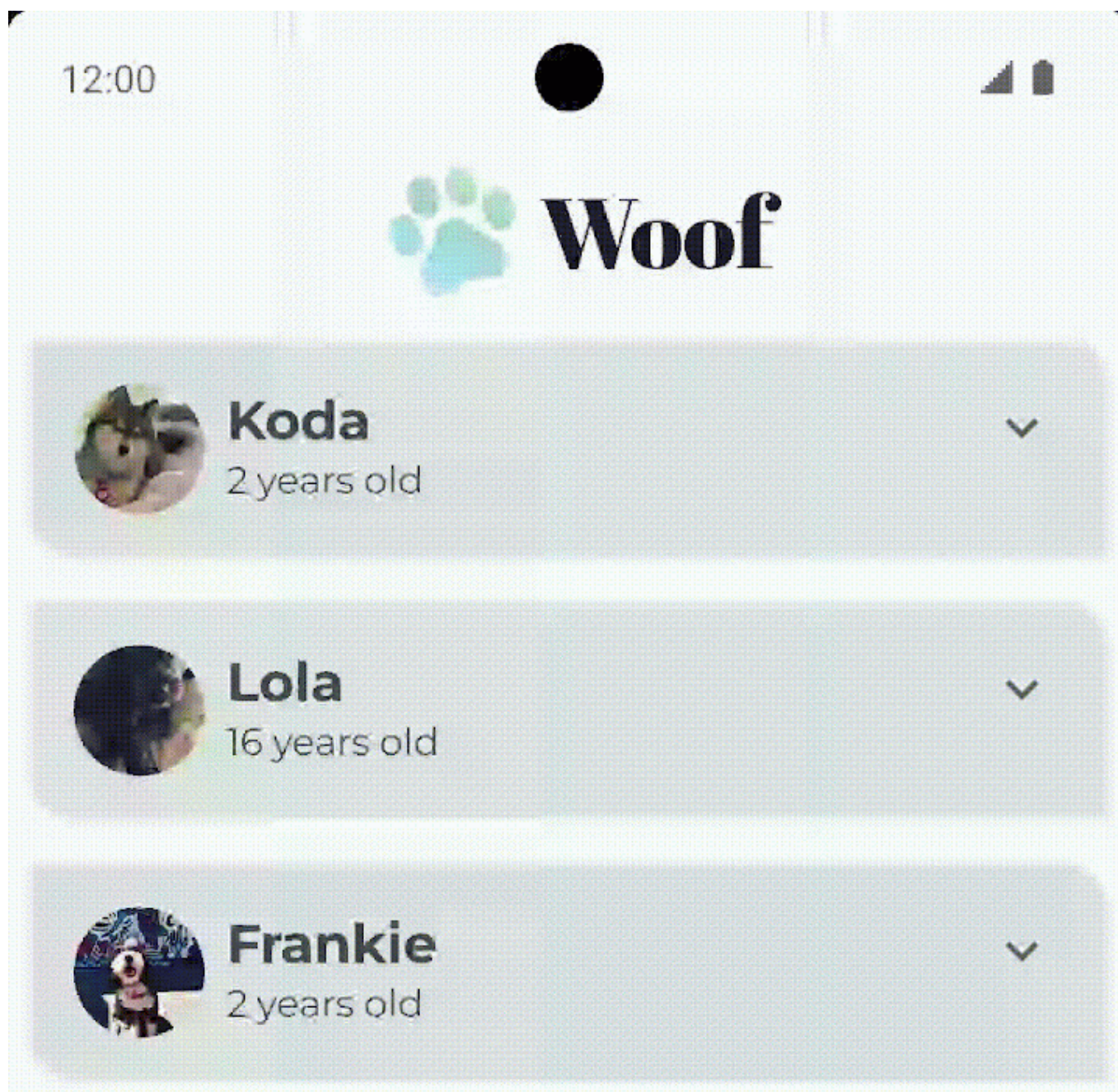


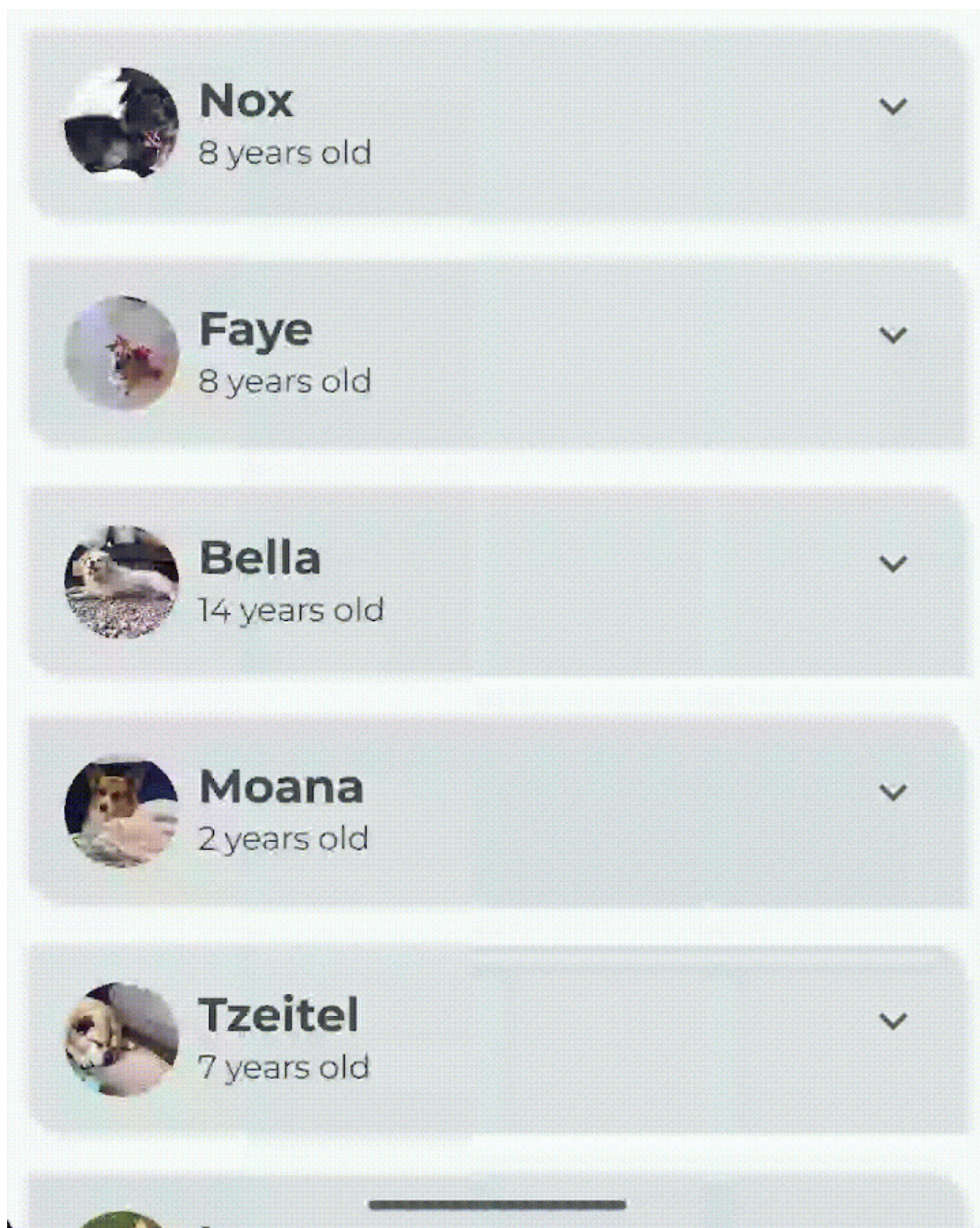
Moana

2 years old



В этом уроке вы добавите анимацию в приложение Woof. Вы добавите информацию о хобби, которая будет отображаться при раскрытии элемента списка. Вы также добавите весеннюю анимацию, чтобы анимировать разворачивающийся элемент списка.





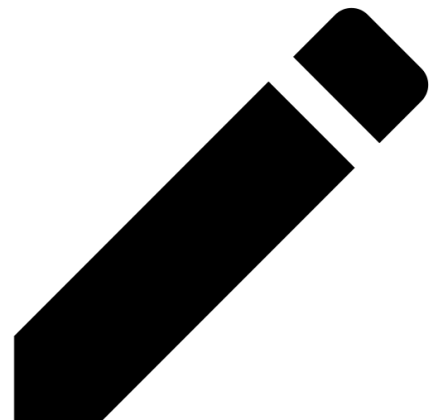
Добавьте иконку Expand More

В этом разделе вы добавите иконки **Expand More** и **Expand Less** в ваше приложение.

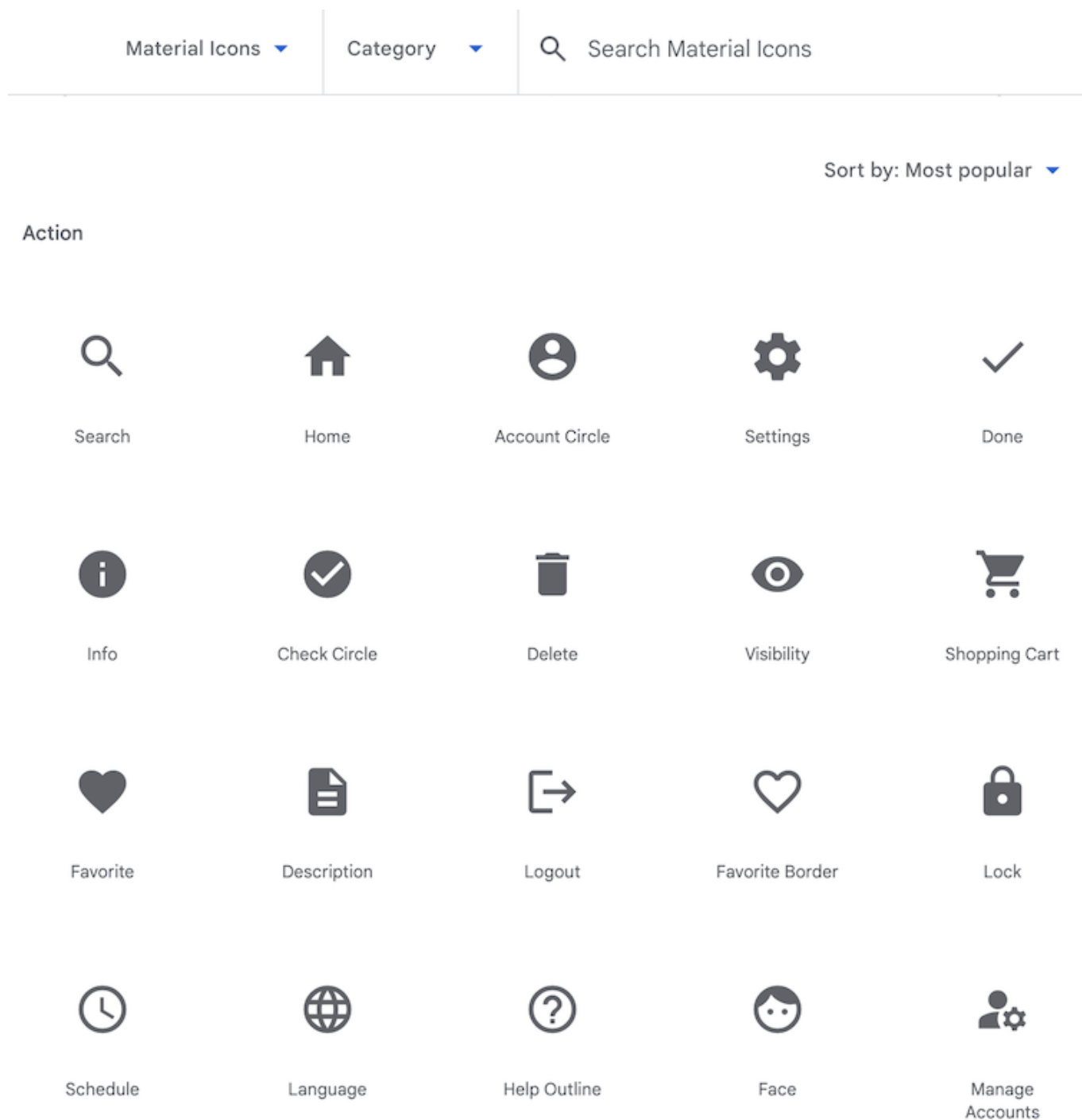


Иконки

Значки - это символы, которые помогают пользователям понять интерфейс, визуально передавая его назначение. Они часто черпают вдохновение в объектах физического мира, с которыми, как ожидается, сталкивался пользователь. При разработке иконок уровень детализации часто снижается до минимума, необходимого для того, чтобы быть знакомым пользователю. Например, карандаш в физическом мире используется для письма, поэтому его иконка обычно обозначает «создать» или «редактировать».



Material Design предлагает множество иконок, распределенных по общим категориям, для большинства ваших нужд.



Добавьте зависимость Gradle

Добавьте зависимость библиотеки `material-icons-extended` в ваш проект. Вы будете использовать иконки `Icons.Filled.ExpandLess` и `Icons.Filled.ExpandMore` из этой библиотеки.

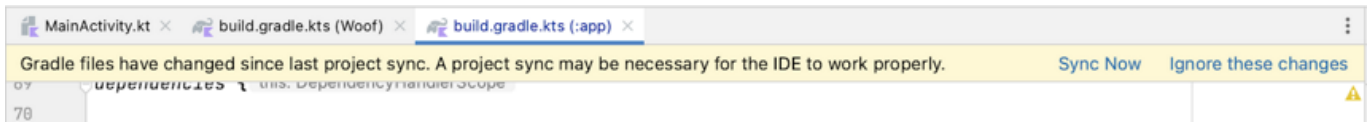
Примечание о зависимостях Gradle: Чтобы добавить зависимость в свой проект, укажите конфигурацию зависимости, такую как `implementation`, в блоке `dependencies` файла `build.gradle.kts` вашего модуля. Когда вы собираете свое приложение, система сборки компилирует библиотечный модуль и упаковывает полученное скомпилированное содержимое в приложение.

На панели **Project** откройте **Gradle Scripts** > `build.gradle.kts` (Module :app).

Прокрутите до конца файл `build.gradle.kts` (Модуль `:app`). В блоке `dependencies{}` добавьте следующую строку:

```
implementation("androidx.compose.material:material-icons-extended")
```

Совет: Когда вы изменяете файлы Gradle, Android Studio может потребоваться импортировать или обновить библиотеки и запустить некоторые фоновые задачи. Android Studio отображает всплывающее окно с предложением синхронизировать проект. Нажмите **Sync Now**.



Добавьте композитную иконку

Добавьте функцию для отображения иконки **Expand More** из библиотеки иконок Material и используйте ее в качестве кнопки.

- В файле `MainActivity.kt` после функции `DogItem()` создайте новую композитную функцию `DogItemButton()`. Передайте в нее булево значение для расширенного состояния, лямбда-выражение для обработчика кнопки `onClick` и необязательный модификатор следующим образом:

```
@Composable
private fun DogItemButton(
    expanded: Boolean,
    onClick: () -> Unit,
    modifier: Modifier = Modifier
) {

}
```

- Внутри функции `DogItemButton()` добавьте составную `IconButton()`, которая принимает именованный параметр `onClick`, лямбду с использованием синтаксиса `trailing lambda`, которая вызывается при нажатии на эту иконку, и необязательный модификатор. Установите значения параметров `IconButton` `onClick` и модификатора равными тем, которые были переданы в `DogItemButton`.

```
@Composable
private fun DogItemButton(
    expanded: Boolean,
    onClick: () -> Unit,
    modifier: Modifier = Modifier
){
    IconButton(
```

```

        onClick = onClick,
        modifier = modifier
    ) {

    }
}

```

- Внутри лямбда-блока `IconButton()` добавьте композитный `Icon` и установите значение-параметр `imageVector` в `Icons.Filled.ExpandMore`. Android Studio выдает предупреждение о параметрах составного `Icon()`, которое вы исправите в следующем шаге.

```

import androidx.compose.material.icons.filled.ExpandMore
import androidx.compose.material.icons.Icons
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton

IconButton(
    onClick = onClick,
    modifier = modifier
) {
    Icon(
        imageVector = Icons.Filled.ExpandMore
    )
}

```

- Добавьте параметр значения `tint` и установите цвет иконки на `MaterialTheme.colorScheme.secondary`. - Добавьте именованный параметр `contentDescription` и установите его в строковый ресурс `R.string.expand_button_content_description`.

```

IconButton(
    onClick = onClick,
    modifier = modifier
){
    Icon(
        imageVector = Icons.Filled.ExpandMore,
        contentDescription =
stringResource(R.string.expand_button_content_description),
        tint = MaterialTheme.colorScheme.secondary
    )
}

```

Отображение иконки

- Отобразите композит `DogItemButton()`, добавив его в макет.
- В начале функции `DogItem()` добавьте переменную для сохранения развернутого состояния элемента списка.

- Установите начальное значение false.

```
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue

var expanded by remember { mutableStateOf(false) }
```

Освежите в памяти функции `remember()` и `mutableStateOf()`: Используйте функцию `mutableStateOf()`, чтобы Compose наблюдал за любыми изменениями значения состояния и запускал рекомпозицию для обновления пользовательского интерфейса.

- Оберните вызов функции `mutableStateOf()` функцией `remember()`, чтобы сохранить значение в Composition при начальной композиции, а сохраненное значение вернуть при рекомпозиции.

Отображение кнопки с иконкой внутри элемента списка.

- В композиции `DogItem()` в конце блока `Row`, после вызова `DogInformation()`, добавьте `DogItemButton()`.
- Передайте в нее расширенное состояние и пустую лямбду для обратного вызова. Вы определите действие `onClick` на следующем шаге.

```
Row(
    modifier = Modifier
        .fillMaxWidth()
        .padding(dimensionResource(R.dimen.padding_small))
) {
    DogIcon(dog.imageResourceId)
    DogInformation(dog.name, dog.age)
    DogItemButton(
        expanded = expanded,
        onClick = { /*TODO*/ }
    )
}
```

- Проверьте функцию `WoofPreview()` в панели Design.





Koda



2 years old



Lola



16 years old



Frankie



2 years old



Nox



8 years old



Faye



8 years old



Bella



14 years old



Moana



2 years old



Tzeitel



7 years old

- Обратите внимание, что кнопка «Развернуть еще» не выровнена по концу элемента списка. Вы исправите это в следующем шаге.

Выравнивание кнопки «Развернуть»

Чтобы выравнивать кнопку «Развернуть» по концу элемента списка, нужно добавить в макет разделитель с атрибутом `Modifier.weight()`.

Примечание: `Modifier.weight()` устанавливает ширину/высоту элемента пользовательского интерфейса пропорционально его весу, относительно его взвешенных братьев и сестер (других дочерних элементов в строке или столбце).

Пример: Рассмотрим три дочерних элемента в строке с весами 1f, 1f и 2f. В этом случае все дочерние элементы имеют назначенные веса. Свободное место в строке распределяется пропорционально указанному значению веса, при этом больше свободного места достается дочерним элементам с большим значением веса. Дочерние элементы распределяют вес, как показано ниже:



В приведенном выше ряду первый дочерний компоуемый элемент имеет $\frac{1}{4}$ ширины ряда, второй также имеет $\frac{1}{4}$ ширины ряда, а третий - $\frac{1}{2}$ ширины ряда. Если у дочерних элементов нет назначенных весов (`weight` - необязательный параметр), то высота/ширина дочернего композитного элемента по умолчанию будет соответствовать параметру `wrap content` (обертывание содержимого того, что находится внутри элемента пользовательского интерфейса).

Примечание о float значениях: float значения в Kotlin - это десятичные числа, обозначаемые буквой `f` или `F` в конце числа.

В приложении **Woof** каждая строка списка содержит изображение собаки, информацию о ней и кнопку «Развернуть». Чтобы правильно выравнивать значок кнопки, перед кнопкой «Развернуть еще» добавляется композит `Spacer` с весом **1f**. Поскольку `padding` является единственным взвешенным дочерним элементом в строке, она заполнит пространство, оставшееся в строке после измерения ширины других невзвешенных дочерних элементов.



- Добавьте разделитель в строку элемента списка В `DogItem()`, между `DogInformation()` и `DogItemButton()`, добавьте спейсер. Передайте модификатор с весом(1f). Модификатор `weight()` заставляет спейсер заполнить пространство, оставшееся в строке.

```
import androidx.compose.foundation.layout.Spacer

Row(
    modifier = Modifier
        .fillMaxWidth()
        .padding(dimensionResource(R.dimen.padding_small))
) {
    DogIcon(dog.imageResourceId)
    DogInformation(dog.name, dog.age)
    Spacer(modifier = Modifier.weight(1f))
    DogItemButton(
        expanded = expanded,
        onClick = { /*TODO*/ }
    )
}
```

- Проверьте функцию `WoofPreview()` в панели Design. Обратите внимание, что кнопка «Развернуть еще» теперь выровнена по концу элемента списка.

WoofPreview

**Koda**

2 years old

**Lola**

16 years old

**Frankie**

2 years old

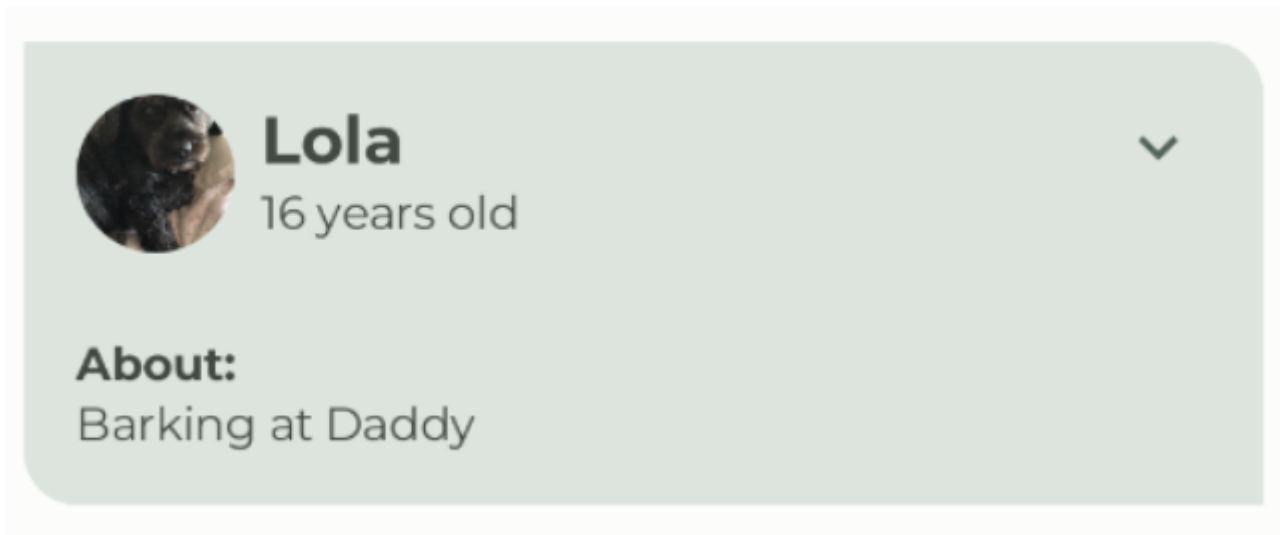
**Nox**

8 years old



Добавьте композит для отображения хобби

В этом задании вы добавите Text composables для отображения информации о хобби собаки.



- Создайте новую составную функцию `DogHobby()`, которая принимает строковый идентификатор ресурса «Хобби» собаки и необязательный модификатор.

```
@Composable
fun DogHobby(
    @StringRes dogHobby: Int,
    modifier: Modifier = Modifier
) {
}
```

- Внутри функции `DogHobby()` создайте столбец и передайте в него модификатор, переданный в `DogHobby()`.

```
@Composable
fun DogHobby(
    @StringRes dogHobby: Int,
    modifier: Modifier = Modifier
){
    Column(
        modifier = modifier
    ) {

    }
}
```

- Внутри блока `Column` добавьте два составных элемента `Text` - один для отображения текста `About` над информацией о хобби, а другой - для отображения информации о хобби.
- Установите для первого из них текст `about` из файла `strings.xml` и задайте стиль `labelSmall`. Для второго установите текст `dogHobby`, который передается в файл, и установите стиль `bodyLarge`.

```
Column(
    modifier = modifier
```

```

) {
    Text(
        text = stringResource(R.string.about),
        style = MaterialTheme.typography.labelSmall
    )
    Text(
        text = stringResource(dogHobby),
        style = MaterialTheme.typography.bodyLarge
    )
}

```

- В DogItem() составной элемент DogHobby() будет располагаться ниже строки, содержащей DogIcon(), DogInformation(), Spacer() и DogItemButton(). Для этого оберните Строку столбцом, чтобы хобби можно было добавить под Строкой.

```

Column() {
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .padding(dimensionResource(R.dimen.padding_small))
    ) {
        DogIcon(dog.imageResourceId)
        DogInformation(dog.name, dog.age)
        Spacer(modifier = Modifier.weight(1f))
        DogItemButton(
            expanded = expanded,
            onClick = { /*TODO*/ }
        )
    }
}

```

- Добавьте DogHobby() после Row в качестве второго дочернего элемента Column. Передайте dog.hobbies, который содержит уникальное хобби переданной собаки и модификатор с паддингом для композита DogHobby().

```

Column() {
    Row() {
        ...
    }
    DogHobby(
        dog.hobbies,
        modifier = Modifier.padding(
            start = dimensionResource(R.dimen.padding_medium),
            top = dimensionResource(R.dimen.padding_small),
            end = dimensionResource(R.dimen.padding_medium),
            bottom = dimensionResource(R.dimen.padding_medium)
        )
    )
}

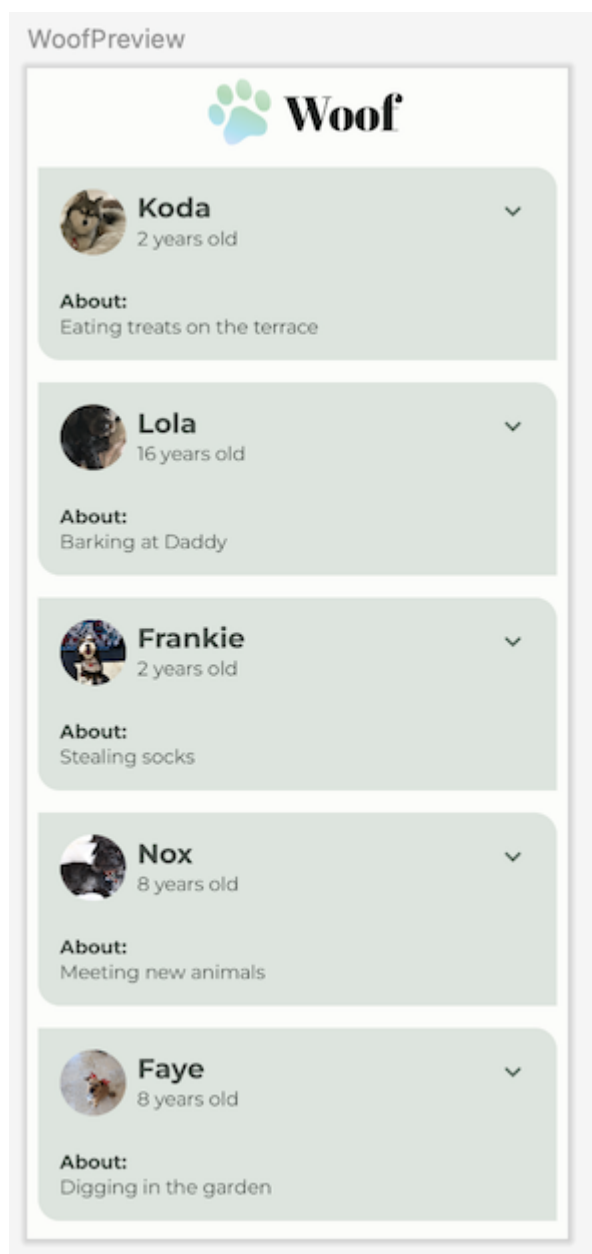
```


Заметка: о собачьих увлечениях: Информация о собачьих увлечениях для всех собак уже предоставлена вам как часть стартового кода. Чтобы увидеть это в своем коде, откройте файл data/Dog.kt, обратите внимание на список собак, предварительно заполненный информацией о них.

Полная функция DogItem() должна выглядеть следующим образом:

```
@Composable
fun DogItem(
    dog: Dog,
    modifier: Modifier = Modifier
) {
    var expanded by remember { mutableStateOf(false) }
    Card(
        modifier = modifier
    ) {
        Column() {
            Row(
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(dimensionResource(R.dimen.padding_small))
            ) {
                DogIcon(dog.imageResourceId)
                DogInformation(dog.name, dog.age)
                Spacer(Modifier.weight(1f))
                DogItemButton(
                    expanded = expanded,
                    onClick = { /*TODO*/ },
                )
            }
            DogHobby(
                dog.hobbies,
                modifier = Modifier.padding(
                    start = dimensionResource(R.dimen.padding_medium),
                    top = dimensionResource(R.dimen.padding_small),
                    end = dimensionResource(R.dimen.padding_medium),
                    bottom = dimensionResource(R.dimen.padding_medium)
                )
            )
        }
    }
}
```

- Посмотрите на функцию WoofPreview() в панели Design. Обратите внимание на отображение хобби собаки.



Показывать или скрывать хобби по нажатию кнопки

В вашем приложении есть кнопка «Развернуть больше» для каждого элемента списка, но она еще ничего не делает! В этом разделе вы добавите возможность скрывать или раскрывать информацию о хобби, когда пользователь нажимает на кнопку «Развернуть больше».

- В составной функции `DogItem()`, в вызове функции `DogItemButton()`, определите лямбда-выражение `onClick()`, измените значение состояния `expanded` boolean на `true`, когда кнопка нажата, и измените его обратно на `false`, если кнопка нажата снова.

```
DogItemButton(  
    expanded = expanded,  
    onClick = { expanded = !expanded }  
)
```

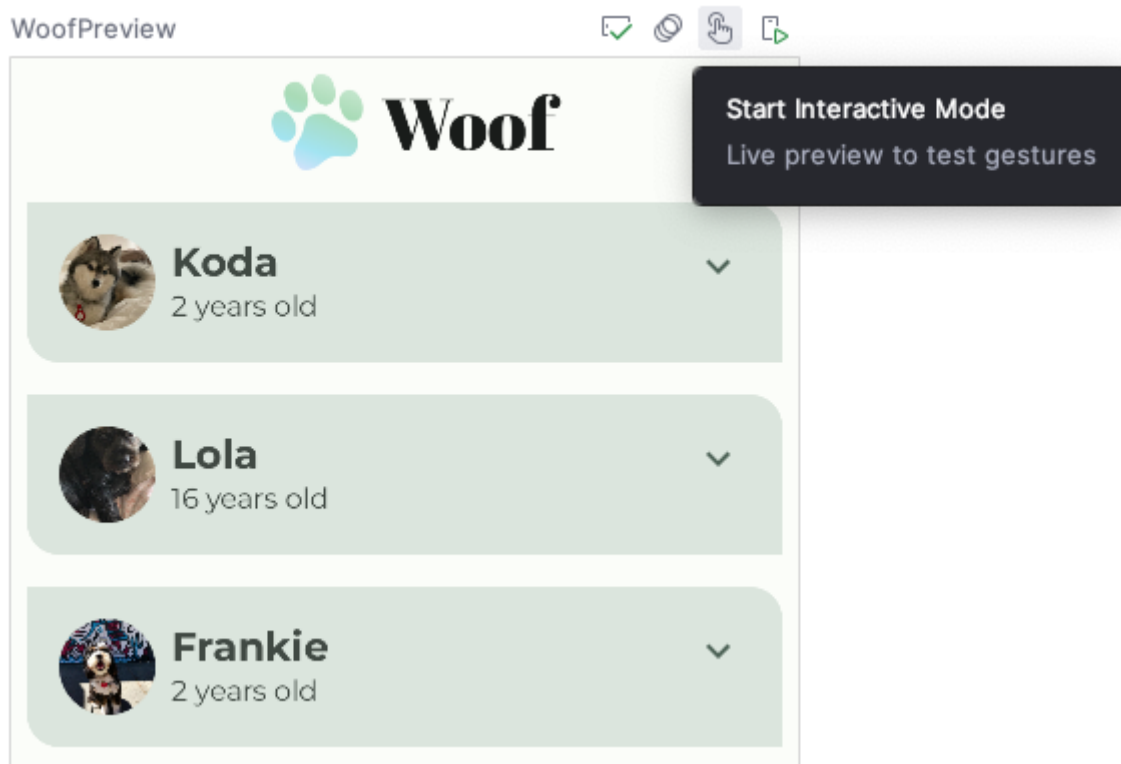
Примечание: Логический оператор NOT (!) возвращает отрицаемое значение булева выражения. Например, если `expanded` равно `true`, то `!expanded` будет равно `false`.

- В функции `DogItem()` оберните вызов функции `DogHobby()` проверкой `if` для расширенного булева выражения.

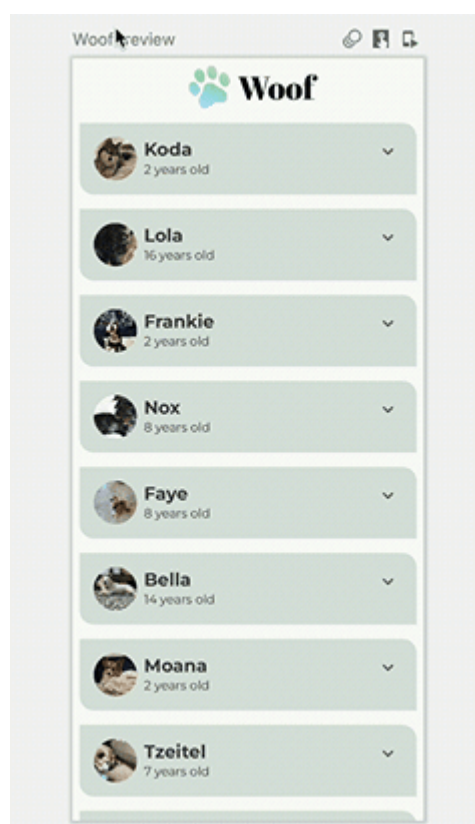
```
@Composable
fun DogItem(
    dog: Dog,
    modifier: Modifier = Modifier
) {
    var expanded by remember { mutableStateOf(false) }
    Card(
        ...
    ) {
        Column(
            ...
        ) {
            Row(
                ...
            ) {
                ...
            }
            if (expanded) {
                DogHobby(
                    dog.hobbies, modifier = Modifier.padding(
                        start = dimensionResource(R.dimen.padding_medium),
                        top = dimensionResource(R.dimen.padding_small),
                        end = dimensionResource(R.dimen.padding_medium),
                        bottom = dimensionResource(R.dimen.padding_medium)
                    )
                )
            }
        }
    }
}
```

- Теперь информация о хобби собаки отображается только в том случае, если значение параметра `expanded` равно `true`.

Предварительный просмотр может показать вам, как выглядит пользовательский интерфейс, но вы также можете взаимодействовать с ним. Чтобы взаимодействовать с предварительным просмотром пользовательского интерфейса, наведите курсор на текст `WoofPreview` в панели дизайна, а затем нажмите кнопку Интерактивный режим в правом верхнем углу панели дизайна. Это запустит предварительный просмотр в интерактивном режиме.



Взаимодействуйте с предварительным просмотром, нажав кнопку «Развернуть еще». Обратите внимание, что информация о хобби собаки скрывается и раскрывается, когда вы нажимаете кнопку «Развернуть».



Примечание: Чтобы остановить интерактивный режим, нажмите кнопку Остановить интерактивный режим в левом верхнем углу панели предварительного просмотра.

■ Stop Interactive Mode

Up-to-date ✓

WoofPreview



Обратите внимание, что значок кнопки «Развернуть больше» остается неизменным, когда элемент списка развернут. Для улучшения пользовательского опыта вы измените значок так, чтобы `ExpandMore` отображала стрелку вниз, а `ExpandLess` - стрелку вверх.

- В функции `DogItemButton()` добавьте оператор `if`, который обновляет значение `imageVector` на основе расширенного состояния следующим образом:

```
import androidx.compose.material.icons.filled.ExpandLess

@Composable
private fun DogItemButton(
    ...
) {
    IconButton(onClick = onClick) {
        Icon(
            imageVector = if (expanded) Icons.Filled.ExpandLess else
            Icons.Filled.ExpandMore,
            ...
        )
    }
}
```

Обратите внимание, как вы написали `if-else` в предыдущем фрагменте кода.

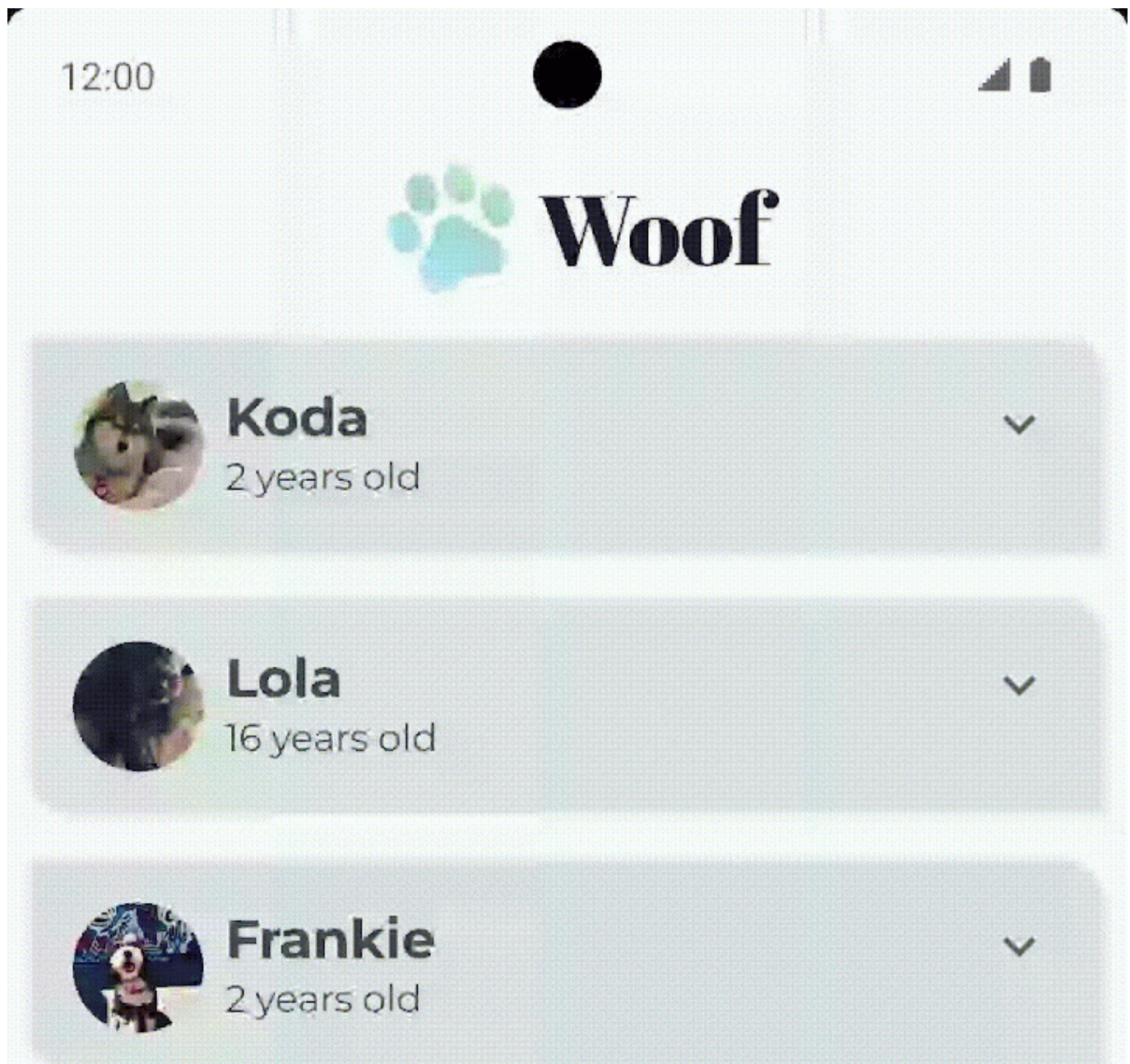

```
if (expanded) Icons.Filled.ExpandLess else Icons.Filled.ExpandMore
```

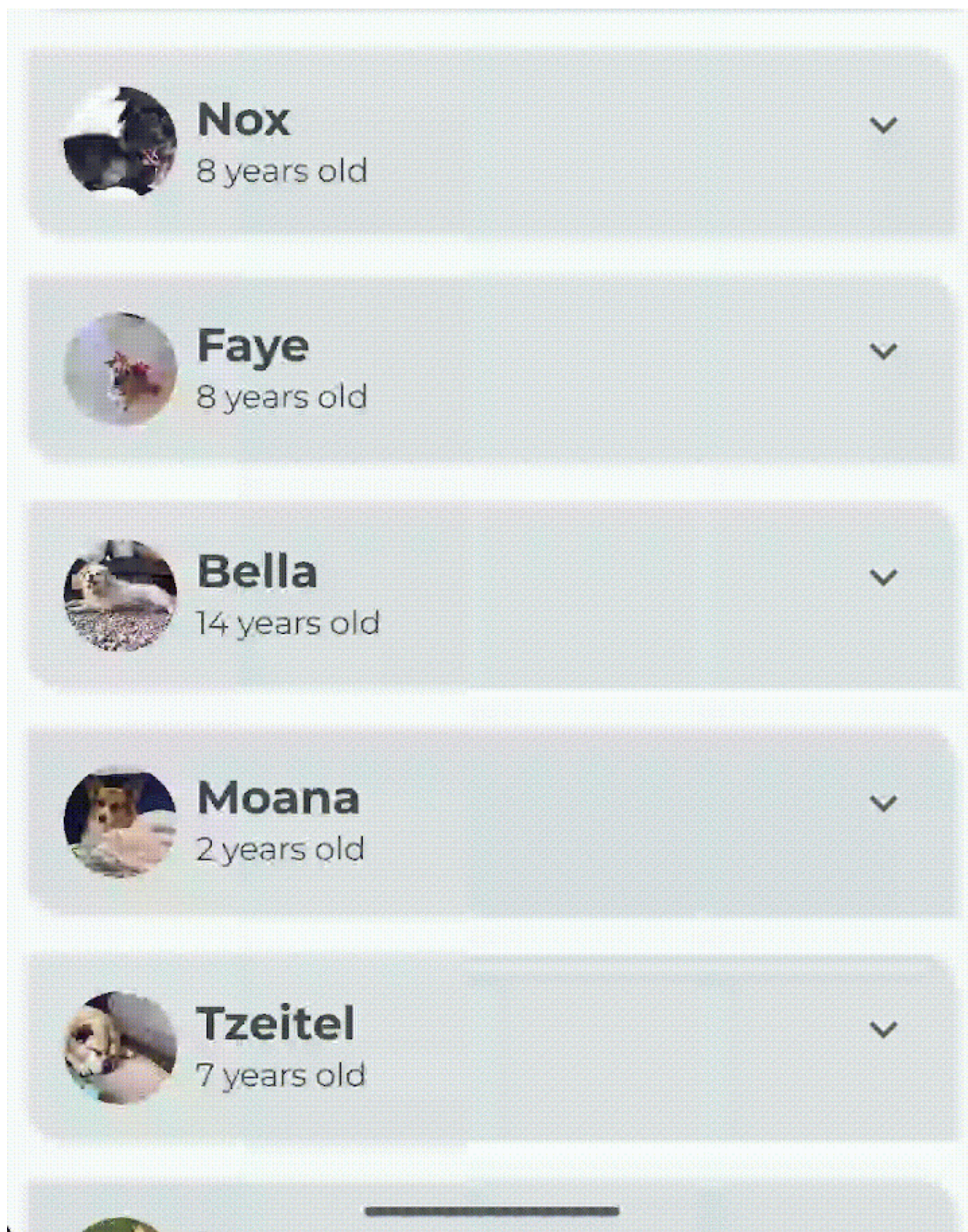
Это то же самое, что и использование фигурных скобок { } в следующем коде:

```
if (expanded) {  
  `Icons.Filled.ExpandLess`  
} else {  
  `Icons.Filled.ExpandMore`  
}
```

Фигурные скобки необязательны, если оператор if-else состоит из одной строки кода.

- Запустите приложение на устройстве или в эмуляторе, или снова используйте интерактивный режим в предварительном просмотре. Обратите внимание, что значок чередуется между значками **ExpandMore** и **ExpandLess**.





Примечание об интерактивном режиме: Интерактивный режим позволяет вам взаимодействовать с предварительным просмотром так же, как вы взаимодействовали бы на устройстве. Однако режим предварительного просмотра не заменяет запуск приложения на устройстве для тестирования.

Когда вы развернули элемент списка, заметили ли вы резкое изменение высоты? Резкое изменение высоты не похоже на отполированное приложение. Чтобы решить эту проблему, вы добавите в приложение анимацию.

Добавьте анимацию

Анимации могут добавлять визуальные подсказки, которые уведомляют пользователей о том, что происходит в вашем приложении. Они особенно полезны, когда пользовательский интерфейс меняет состояние, например, когда загружается новый контент или становятся доступны новые действия. Анимации также могут придать приложению полированный вид.

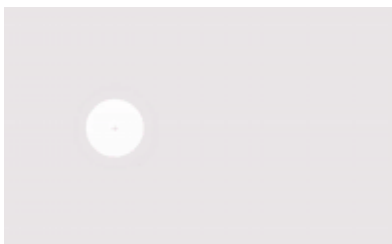
В этом разделе вы добавите анимацию для изменения высоты элемента списка.

Пружинная анимация

Пружинная анимация - это анимация, основанная на физике, которая управляется силой пружины. При пружинной анимации величина и скорость движения рассчитываются в зависимости от приложенной силы пружины.

Например, если вы перетащите значок приложения по экрану, а затем отпустите его, подняв палец, значок под действием невидимой силы переместится в исходное положение.

Следующая анимация демонстрирует эффект пружины. Как только палец отпускается от значка, значок отпрыгивает назад, имитируя пружину.

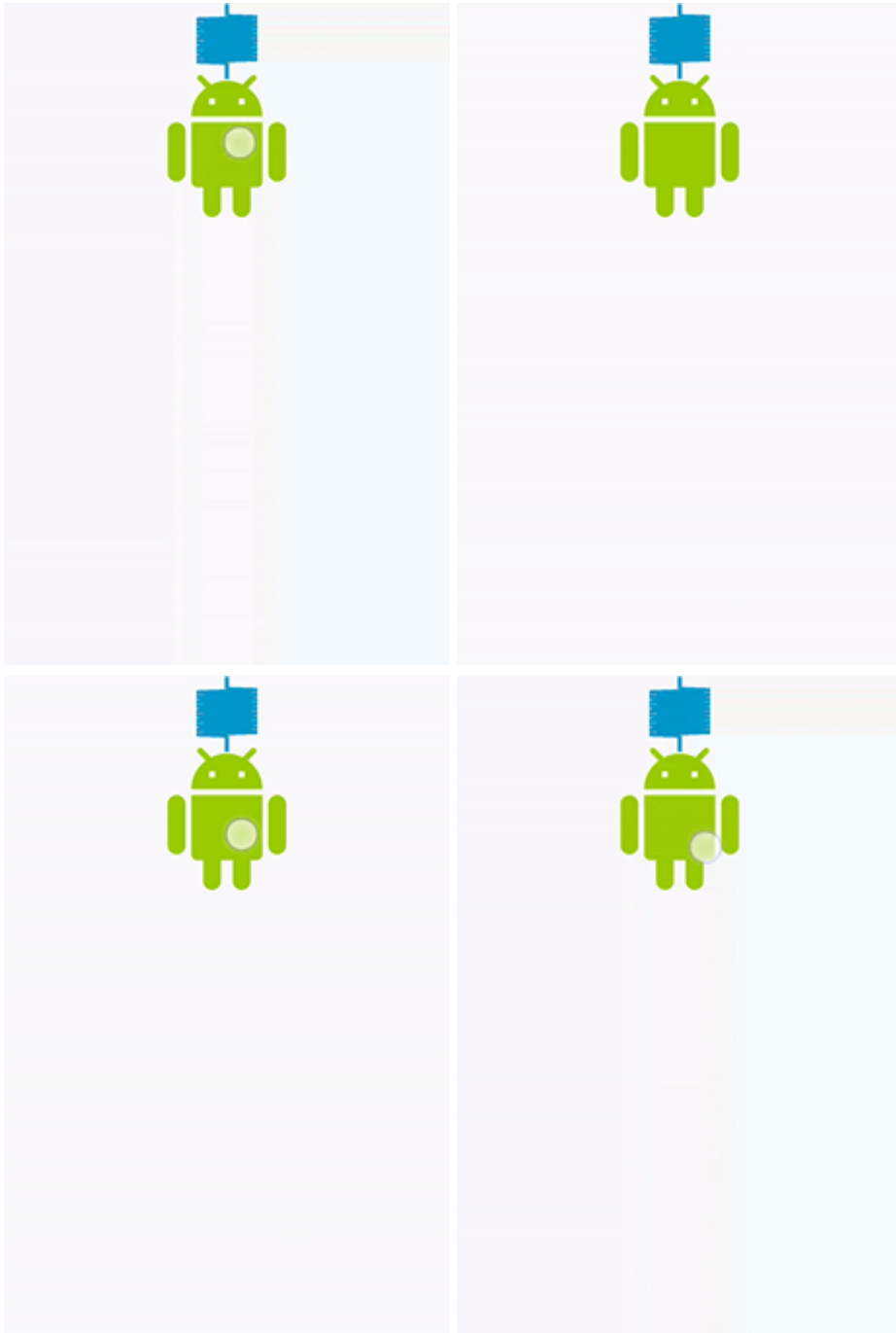


Spring effect

Сила пружины определяется двумя следующими свойствами:

- Коэффициент демпфирования: Отказоустойчивость пружины.
- Уровень жесткости: Жесткость пружины, то есть то, насколько быстро пружина движется к концу.

Ниже приведено несколько примеров анимации [различные коэффициенты демпфирования и уровни жесткости](#).



- Посмотрите на вызов функции `DogHobby()` в композитной функции `DogItem()`. Информация о хобби собаки включается в композицию, основываясь на расширенном булевом значении. Высота элемента списка меняется в зависимости от того, видна или скрыта информация о хобби. В настоящее время переход осуществляется нечетко. В этом разделе вы воспользуетесь модификатором `animateContentSize`, чтобы добавить более плавный переход между расширенным и нерасширенным состояниями.

```
// No need to copy over
@Composable
fun DogItem(...) {
    ...
    if (expanded) {
        DogHobby(
            dog.hobbies,
            modifier = Modifier.padding(
```



```

        start = dimensionResource(R.dimen.padding_medium),
        top = dimensionResource(R.dimen.padding_small),
        end = dimensionResource(R.dimen.padding_medium),
        bottom = dimensionResource(R.dimen.padding_medium)
    )
}
}
}

```

В файле MainActivity.kt в функции DogItem() добавьте параметр-модификатор для макета Column.

```

@Composable
fun DogItem(
    dog: Dog,
    modifier: Modifier = Modifier
) {
    ...
    Card(
        ...
    ) {
        Column(
            modifier = Modifier
        ){
            ...
        }
    }
}

```

- Сцепите этот модификатор с модификатором animateContentSize, чтобы анимировать изменение размера (высоты элемента списка).

```

import androidx.compose.animation.animateContentSize

Column(
    modifier = Modifier
        .animateContentSize()
)

```

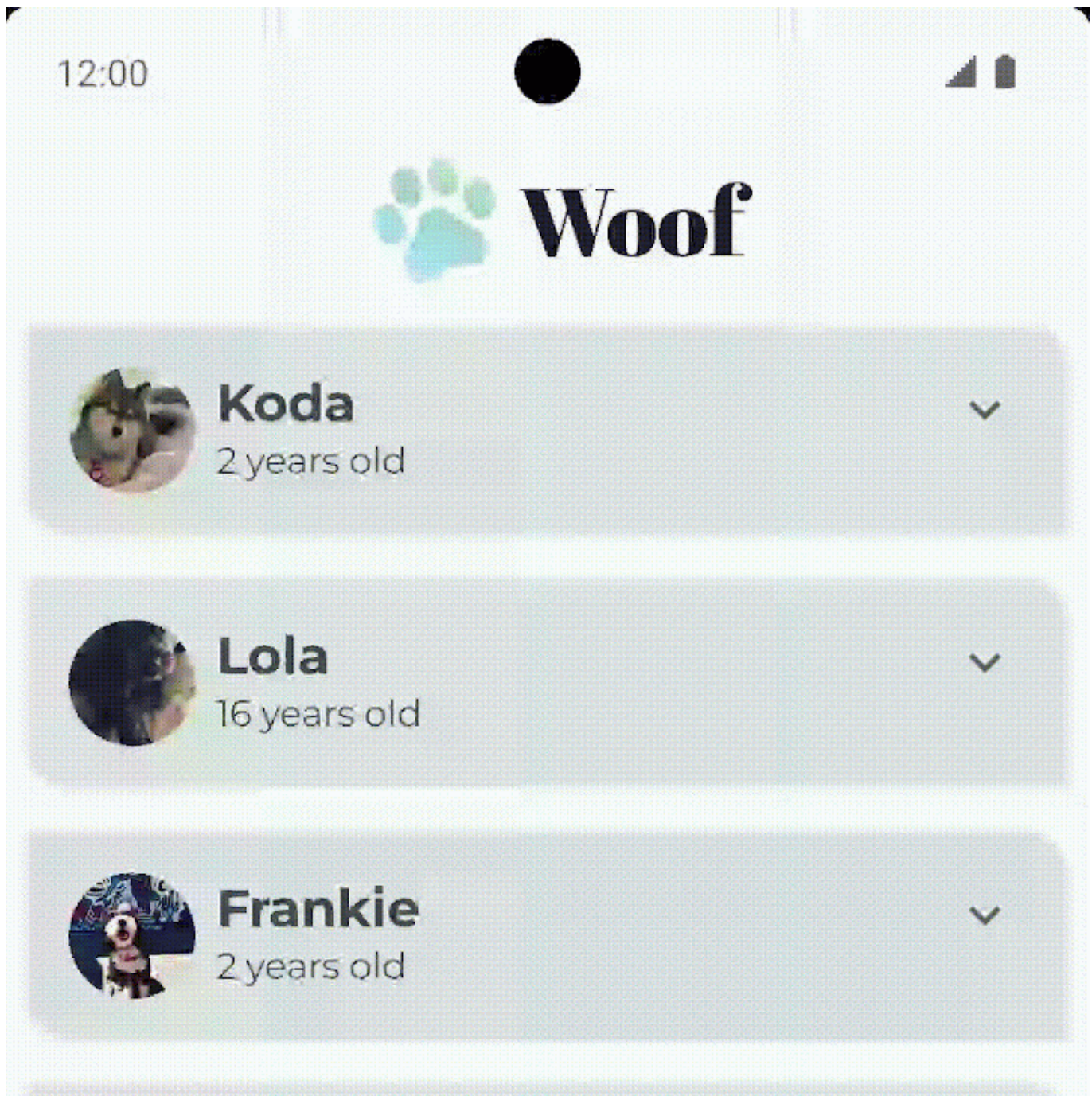
В текущей реализации вы анимируете высоту элементов списка в своем приложении. Но анимация настолько тонкая, что ее трудно заметить при запуске приложения. Чтобы решить эту проблему, используйте дополнительный параметр animationSpec, который позволит вам настроить анимацию.

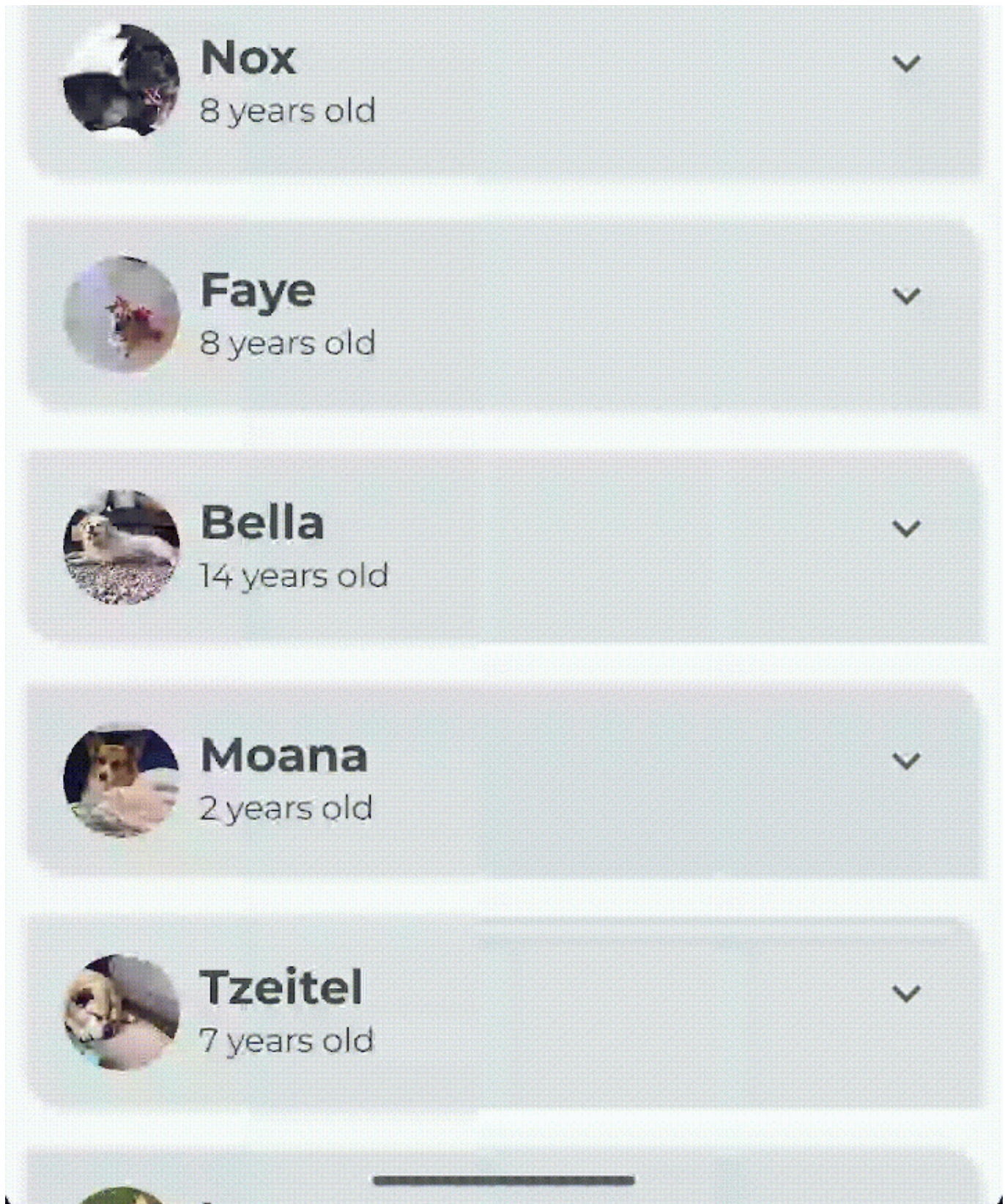
- Для **Woof** анимация плавно приближается и удаляется без скачков. Чтобы добиться этого, добавьте параметр animationSpec в вызов функции animateContentSize(). Установите для анимации пружину с параметром DampingRatioNoBouncy, чтобы не было отскока, и параметр StiffnessMedium, чтобы сделать пружину немного жестче.

```
import androidx.compose.animation.core.Spring
import androidx.compose.animation.core.spring

Column(
    modifier = Modifier
        .animateContentSize(
            animationSpec = spring(
                dampingRatio = Spring.DampingRatioNoBouncy,
                stiffness = Spring.StiffnessMedium
            )
        )
)
```

- Проверьте функцию `WoofPreview()` в панели Design и воспользуйтесь интерактивным режимом или запустите приложение на эмуляторе или устройстве, чтобы увидеть анимацию в действии.





Вы сделали это! Наслаждайтесь красивым приложением с анимацией.

7. (Необязательно) Поэкспериментируйте с другими анимациями

`animate*AsState`

- Функции `animate*AsState()` - это один из самых простых API анимации в Compose для анимации одного значения. Вы указываете только конечное значение (или целевое значение), и API запускает анимацию от текущего значения до указанного конечного значения.

Compose предоставляет функции `animate*AsState()` для `Float`, `Color`, `Dp`, `Size`, `Offset` и `Int`, и это лишь некоторые из них. Вы можете легко добавить поддержку других типов данных с помощью функции `animateValueAsState()`, которая принимает общий тип.

Попробуйте использовать функцию `animateColorAsState()` для изменения цвета при раскрытии элемента списка.

В функции `DogItem()` объявите цвет и делегируйте его инициализацию функции `animateColorAsState()`.

```
import androidx.compose.animation.animateColorAsState

@Composable
fun DogItem(
    dog: Dog,
    modifier: Modifier = Modifier
) {
    var expanded by remember { mutableStateOf(false) }
    val color by animateColorAsState()
    ...
}
```

- Установите именованный параметр `targetValue` в зависимости от расширенного булевого значения. Если элемент списка расширен, установите для него третичный `colorContainer`. В противном случае установите для него цвет `primaryContainer`.

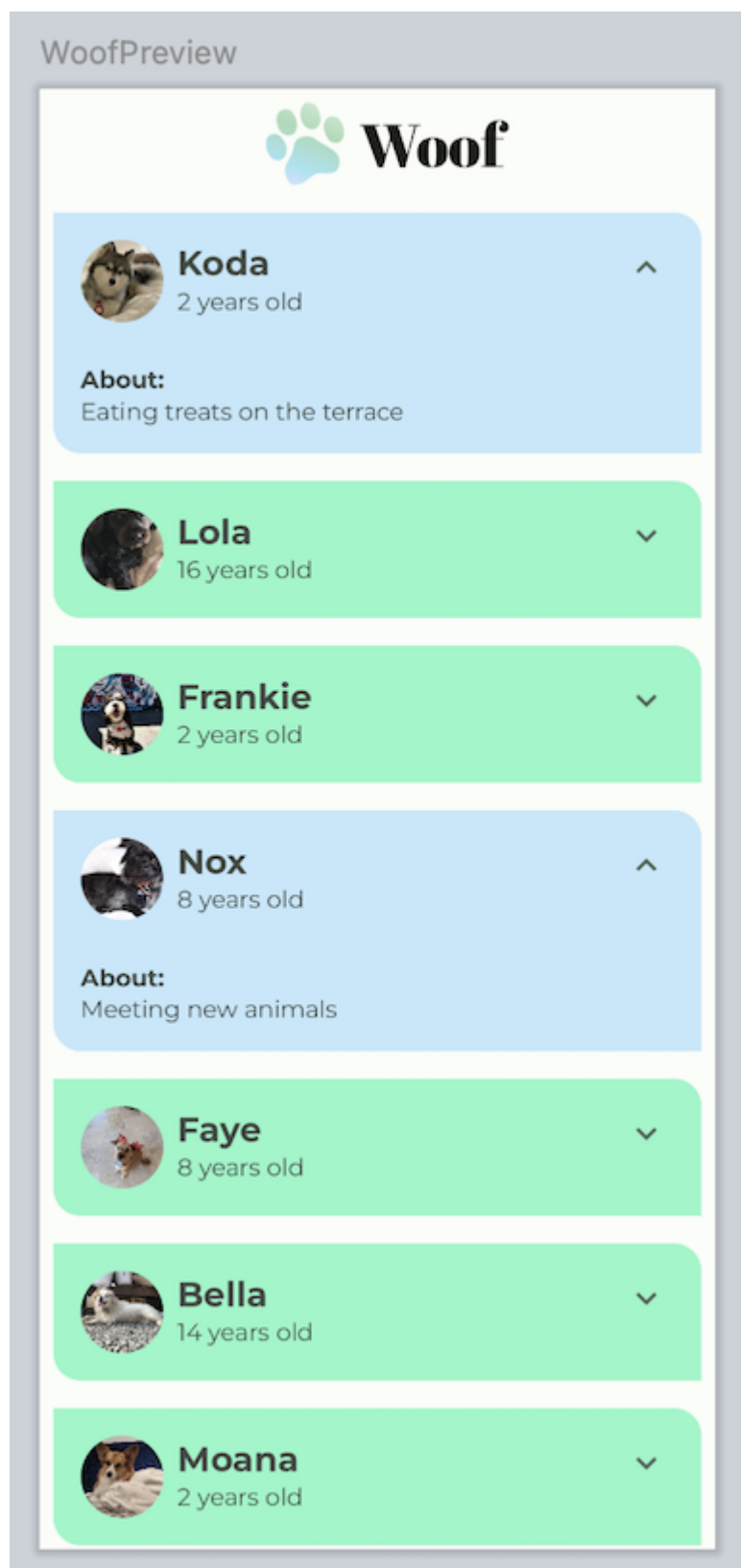
```
import androidx.compose.animation.animateColorAsState

@Composable
fun DogItem(
    dog: Dog,
    modifier: Modifier = Modifier
) {
    var expanded by remember { mutableStateOf(false) }
    val color by animateColorAsState(
        targetValue = if (expanded) MaterialTheme.colorScheme.tertiaryContainer
        else MaterialTheme.colorScheme.primaryContainer,
    )
    ...
}
```

- Установите цвет в качестве модификатора фона для столбца.

```
@Composable
fun DogItem(
    dog: Dog,
    modifier: Modifier = Modifier
) {
    ...
    Card(
        ...
    ) {
        Column(
            modifier = Modifier
                .animateContentSize(
                    ...
                )
        )
        .background(color = color)
    } {...}
}
```

- Посмотрите, как меняется цвет при расширении элемента списка. Элементы списка без расширения имеют цвет `primaryContainer`, а элементы списка с расширением - цвет `tertiaryContainer`.



Заключение

Вы добавили кнопку для скрытия и раскрытия информации о собаке. Вы улучшили пользовательский опыт с помощью анимации. Вы также узнали, как использовать интерактивный режим в панели дизайна. Вы также можете попробовать другой тип анимации Jetpack Compose.